

AI Assisted Coding

Assignment 3.5

Name: Nihaal Krishna

Hallticket: 2303A52498

Task-01

Prompt:

Write a Python function to determine whether a given year is a leap year or not.

Include multiple test years inside the program to demonstrate the output.

```
def is_leap_year(year):  
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):  
        return True  
    return False  
test_years = [1999, 2000, 2004, 1900, 2024]  
for year in test_years:  
    if is_leap_year(year):  
        print(f"{year} is a Leap Year")  
    else:  
        print(f"{year} is Not a Leap Year")
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe  
1999 is Not a Leap Year  
2000 is a Leap Year  
2004 is a Leap Year  
1900 is Not a Leap Year  
2024 is a Leap Year
```

Task-02

Prompt: Write a Python function to find the Greatest Common Divisor (GCD) of two numbers.

Example:

Input: 24, 36

Output: 12

The function should take two integers as input and return their GCD.

```
def gcd(a, b):  
    while b != 0:  
        a, b = b, a % b  
    return abs(a)  
  
if __name__ == "__main__":  
    test_pairs = [(24, 36), (81, 27), (150, 35), (-48, 18)]  
  
    for num1, num2 in test_pairs:  
        print(f"GCD of {num1} and {num2}: {gcd(num1, num2)}")
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe  
GCD of 24 and 36: 12  
GCD of 81 and 27: 27  
GCD of 150 and 35: 5  
GCD of -48 and 18: 6
```

Task 03

prompt:

- # Create a Python function that calculates the Least Common Multiple of two given integers.
- # Use the mathematical relationship between GCD and LCM.
- # Demonstrate the function with new test cases such as: (8, 12), (9, 15), and (14, 20).
- # The function should return the LCM value.

CODE:

```
def lcm(a, b):
    # Helper function to compute GCD using Euclidean algorithm
    def gcd(x, y):
        # Continue until remainder becomes zero
        while y != 0:
            x, y = y, x % y
        # Return absolute value to handle negative inputs
        return abs(x)

    # LCM formula using GCD
    # LCM(a, b) = |a * b| / GCD(a, b)
    return abs(a * b) // gcd(a, b)

# Example test cases to verify correctness
if __name__ == "__main__":
    test_pairs = [(8, 12), (9, 15), (14, 20)]

    for num1, num2 in test_pairs:
        print(f"LCM of {num1} and {num2}: {lcm(num1, num2)}")
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
LCM of 8 and 12: 24
LCM of 9 and 15: 45
LCM of 14 and 20: 140
```

Task 4 :

“Write a Python program that takes a binary number in string format and converts it into its equivalent decimal number.

The program should first validate whether the given input is a proper binary string (containing only 0s and 1s).

Display an appropriate message for invalid inputs.

Test the program with several examples including both valid and invalid binary values.”

```
def binary_to_decimal(binary_string):
    # Check if the input is empty
    if len(binary_string) == 0:
        return "Invalid binary input"

    # Validate that the string contains only 0s and 1s
    for char in binary_string:
        if char not in ('0', '1'):
            return "Invalid binary input"

    # Convert binary to decimal manually
    decimal_value = 0
    power = 0

    for digit in reversed(binary_string):
        decimal_value += int(digit) * (2 ** power)
        power += 1

    return decimal_value

# Test cases (valid and invalid inputs)
test_values = ["1010", "1111", "10001", "10201", "abc", "", "110010"]

for value in test_values:
    result = binary_to_decimal(value)
    print(f"Input: {value} -> Output: {result}")
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Input: 1010 -> Output: 10
Input: 1111 -> Output: 15
Input: 10001 -> Output: 17
Input: 10201 -> Output: Invalid binary input
Input: abc -> Output: Invalid binary input
Input: -> Output: Invalid binary input
Input: 110010 -> Output: 50
```

Task-5

prompt :

'''Write a Python function that converts a given decimal integer into its binary representation.

Demonstrate the function with sample inputs like 12, 25, and 7.

After each output, print a one-word description indicating the result type.'''

Code:

```
def decimal_to_binary(n):  
    """  
    Convert a non-negative decimal integer to its binary representation.  
    Returns the binary value as a string.  
    """  
    if n == 0:  
        return "0"  
  
    binary = ""  
    while n > 0:  
        binary = str(n % 2) + binary  
        n //= 2  
  
    return binary  
  
# Example test cases  
test_numbers = [12, 25, 7]  
  
for num in test_numbers:  
    result = decimal_to_binary(num)  
    print(f"{num} -> {result} (Binary)")
```

OUTPUT:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe  
12 -> 1100 (Binary)  
25 -> 11001 (Binary)  
7 -> 111 (Binary)  
PS C:\Users\nihaa>
```

Task-06:

Prompt:

Write a Python function to check whether a given number is a Harshad (Niven) number.

Examples:

Input: 20 Output: Harshad Number

Input: 21 Output: Harshad Number

Input: 24 Output: Harshad Number

The function should calculate the sum of digits and check divisibility.

Also test boundary conditions such as single-digit numbers, zero, and negative values to ensure robustness.

CODE:

```
def is_harshad_number(n):
    # Create a function to check whether a number is a Harshad (Niven) number.
    # If the number is zero or negative, immediately return "Not a Harshad Number".

    if n <= 0:
        return "Not a Harshad Number"

    # Compute the sum of digits of the given number.
    digit_sum = sum(int(digit) for digit in str(n))

    # Check whether the number is divisible by the sum of its digits.
    if digit_sum != 0 and n % digit_sum == 0:
        return "Harshad Number"

    # Otherwise, return that the number is not a Harshad number.
    return "Not a Harshad Number"

# Add multiple test cases including valid and invalid inputs.
if __name__ == "__main__":
    test_numbers = [20, 21, 24, 11, 0, -9]

    for num in test_numbers:
        print(f"{num}: {is_harshad_number(num)}")
```

OUTPUT:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
20: Harshad Number
21: Harshad Number
24: Harshad Number
11: Not a Harshad Number
0: Not a Harshad Number
-9: Not a Harshad Number
```

